AI SMART INVENTORY MANAGEMENT SYSTEM

USING MERN STACK

A Project Report Submitted

in Partial Fulfillment of the Requirements for the Subject

PROJECT

of

MASTER OF COMPUTER APPLICATIONS (MCA) – II

SUBMITTED BY:

[STUDENT NAME]

[ENROLLMENT NUMBER]

Under the Guidance of

PROF. HIMANI KHODIFAD

R.B. Institute of Management Studies (RBIMS)

APPROVED BY AICTE, GOVT. OF INDIA, NEW DELHI

Affiliated with Gujarat Technological University, Ahmedabad

Ahmedabad, Gujarat

Academic Year: 2025–26

DECLARATION

I hereby declare that the work presented in this project report entitled “AI Smart Inventory Management System using MERN Stack” submitted in partial fulfillment of the requirements for the subject Project of MCA Semester – II under Gujarat Technological University (GTU) is an authentic record of the work carried out by me during the academic year 2025–26 under the guidance of Prof. Himani Khodifad.

The matter embodied in this report has not been submitted by me for the award of any other degree, diploma, or similar title of this or any other University/Institute.

[STUDENT NAME]

[ENROLLMENT NUMBER]

This is to certify that the above statement made by the student is correct to the best of our knowledge.

Project Guide

Prof. Himani Khodifad

Signature: _____

Date: _____

Place: RBIMS, Ahmedabad

R.B. Institute of Management Studies (RBIMS)

APPROVED BY AICTE, GOVT. OF INDIA, NEW DELHI

Affiliated with Gujarat Technological University, Ahmedabad

CERTIFICATE

This is to certify that the project entitled “AI Smart Inventory Management System using MERN Stack” submitted by the following student of MCA Semester – II of R.B. Institute of Management Studies (RBIMS), affiliated with Gujarat Technological University (GTU), Ahmedabad, is a bonafide record of original project work carried out under the guidance and supervision of Prof. Himani Khodifad during the academic year 2025–26.

[STUDENT NAME]

[ENROLLMENT NUMBER]

This work has not been submitted elsewhere for the award of any degree, diploma, or similar title.

Project Guide

Prof. Himani Khodifad

Signature: _____

Head of Department

Signature: _____

R.B. Institute of Management Studies (RBIMS), Ahmedabad, Gujarat

Date: _____

ABSTRACT

The AI Smart Inventory Management System is a full-stack web application designed to digitise the day-to-day operations of small and medium retail businesses such as grocery (kirana) stores, traders and wholesalers. A large proportion of such businesses continue to manage stock in paper registers, prepare bills manually, and record customer credit in physical ledgers. These practices result in calculation errors, lost records, poor tax compliance and an absence of meaningful business insight.

The proposed system addresses these problems through a single integrated platform built on the MERN stack — MongoDB, Express.js, React and Node.js. It provides real-time inventory tracking, rapid point-of-sale billing with automatic GST computation, server-generated PDF invoices and a UPI "Scan-to-Pay" QR code, a digital customer credit ledger (khata), and supplier management. The application also provides intelligent assistance: demand forecasting and reorder suggestions are computed from sales history, an AI help assistant powered by the Google Gemini API answers natural-language questions about the business, and supplier bills are read automatically using Gemini Vision with a Tesseract.js optical-character-recognition fallback. Scheduled background jobs generate periodic reports and raise smart stock alerts automatically.

The system follows a layered three-tier architecture with validated REST APIs, JWT-based authentication, and precise decimal arithmetic for all monetary calculations. The result is a reliable, extensible and practical system that demonstrates the complete software development life cycle — from requirement analysis and database design to implementation and testing.

ACKNOWLEDGEMENT

The completion of this project has been possible through the support and encouragement of many individuals, and I take this opportunity to express my sincere gratitude to all of them.

First and foremost, I express my heartfelt thanks to my project guide, Prof. Himani Khodifad, for her invaluable guidance, constant encouragement and constructive feedback throughout the course of this project. Her insights and suggestions were instrumental in shaping this work.

I am thankful to the Head of the Department and the entire faculty of the Master of Computer Applications programme at R.B. Institute of Management Studies (RBIMS), Ahmedabad, for providing the academic environment and resources required to carry out this project.

I also extend my gratitude to Gujarat Technological University (GTU) for including a practical project component in the curriculum, which gave me the opportunity to apply theoretical knowledge to a real-world problem.

Finally, I thank my family and friends for their continual support and motivation during the development of this project.

[STUDENT NAME]

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1

1. Introduction

Inventory management is the backbone of every retail business. While large enterprises employ sophisticated Enterprise Resource Planning (ERP) systems, the vast majority of small retailers — grocery stores, traders and wholesalers — continue to depend on manual methods. Stock registers are updated by hand, bills are calculated mentally or on a calculator, and customer credit is maintained in a paper ledger commonly known as a khata.

This project, the AI Smart Inventory Management System, was developed to bring the benefits of modern software and artificial intelligence to such businesses in a form that is affordable, simple and aligned with their actual daily workflow. The system unifies inventory, billing, customer credit, supplier records, analytics and reporting into one web application, and applies AI where it produces direct business value: predicting demand, recommending reorders and automating routine analysis.

1.1 Problem Statement

The existing manual approach followed by most small retail businesses suffers from several interrelated problems:

- **Manual stock-keeping:** stock levels recorded on paper are frequently inaccurate, and shortages are discovered only when a customer asks for an item that is unavailable.
- **Error-prone billing:** manual GST computation leads to incorrect tax amounts, non-compliant invoices and revenue leakage.
- **Unreliable credit records:** paper credit ledgers are easily lost or disputed, and outstanding amounts are difficult to total or follow up.
- **No business insight:** the owner has no data on fast-moving products, seasonal demand, stock shrinkage or profitability.
- **Unsuitable existing software:** commercial solutions are often expensive, overly complex, or not designed around the workflow of a small Indian retail business.

1.2 Innovative Ideas of the Project

The project introduces several ideas that distinguish it from a conventional inventory application:

- **Demand forecasting from sales history** that analyses past sales to predict future demand and recommend what to reorder and when.
- **An AI help assistant** (powered by the Google Gemini API) that answers natural-language questions about the business, with a built-in rule-based fallback when the AI service is unavailable.
- **Automatic bill reading,** allowing the owner to photograph a supplier bill and have products extracted automatically (using Gemini Vision, with a Tesseract.js OCR fallback) instead of typing them by hand.
- **A fully digital khata** that replaces the paper credit ledger and keeps an accurate, permanent, timestamped record of every credit and repayment.
- **Amount-first billing,** where the customer asks for a fixed value of a product (for example "Rs. 50 of sugar") and the system back-calculates the quantity.
- **A UPI "Scan-to-Pay" QR code on the invoice,** generated from the business's UPI ID, enabling the customer to scan and pay instantly.
- **Automation through scheduled jobs** that raise stock alerts and generate reports without any manual effort.

1.3 Project Objective

The principal objectives of the system are:

1. To digitise inventory with real-time stock levels, units of measure, low-stock detection and shrinkage (stock-adjustment) tracking.
2. To provide a fast point-of-sale billing module with automatic GST calculation, PDF invoice generation and a UPI Scan-to-Pay QR code for digital payment.
3. To replace the paper credit ledger with a digital khata that maintains an accurate running balance for every customer.
4. To provide intelligent assistance — demand forecasting and reorder suggestions derived from sales history, and an AI-powered help assistant built on the Google Gemini API.
5. To automate routine work — periodic reports and stock alerts — using scheduled background jobs.
6. To deliver analytics dashboards and exportable reports, including a Tally-compatible export for accountants.

7. To ensure correctness and security through input validation, authenticated APIs and precise decimal arithmetic for money.

1.4 Scope of the Project

The system is a web-based application intended for the owner and staff of a small or medium retail business. Its scope covers the complete operational cycle of such a business: adding stock, selling and billing, collecting payment or credit, and analysing performance.

The application supports product and inventory management, point-of-sale billing with GST, customer and supplier management, a digital credit ledger, AI insights, OCR bill scanning, analytics and automated reporting. It is multi-lingual and designed to be usable by non-technical shopkeepers. The current scope is limited to a single business account per login and operation over an internet connection; multi-branch operation, a dedicated mobile application and offline synchronisation are identified as future work.

1.5 Design and Implementation

The application is implemented on the MERN stack and follows a clean separation between the presentation, application and data layers. On the server, functionality is divided into independent modules, each exposed as a group of REST endpoints and backed by a controller and a service that contains the business logic. On the client, each module corresponds to one or more React pages that consume those endpoints. The principal modules of the system are shown in Figure 1.

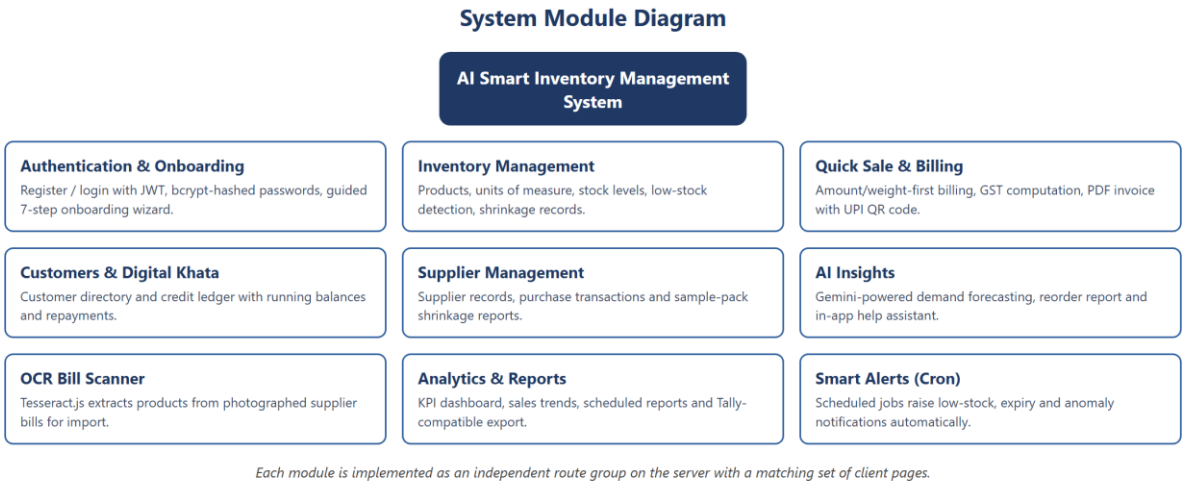

Figure 1 — System Module Diagram

Every business operation updates all related data in a single coherent workflow. When a sale is completed the stock is reduced, the invoice is generated, the payment is recorded and — if the sale is on credit — the customer's ledger is updated. The accumulated data subsequently feeds the analytics and AI layer. This core data flow is illustrated in Figure 2.

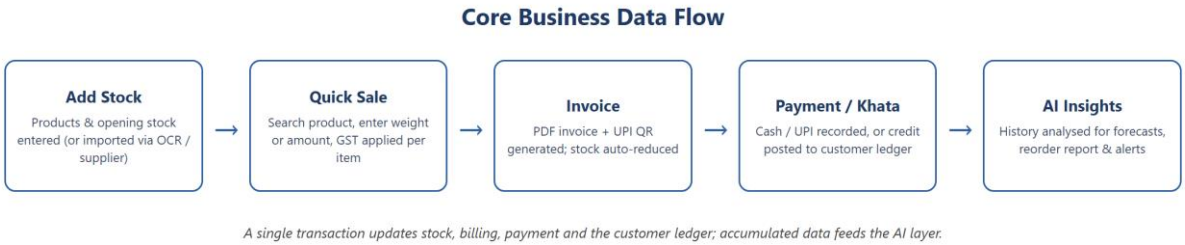

Figure 2 — Core Business Data Flow

1.6 Three-Tier Architecture

The system is organised into a classic three-tier architecture, which separates the user interface, the business logic and the data storage into independent layers. This separation

improves maintainability, allows each layer to be developed and tested in isolation, and makes the system easier to scale and extend.

- **Presentation tier:** a React single-page application that runs in the browser and renders the user interface.
- **Application tier:** a Node.js and Express REST API that authenticates requests, validates input and executes the business logic through a controller-and-service structure.
- **Data tier:** a MongoDB database accessed through the Mongoose object-document mapper, which stores all persistent data.

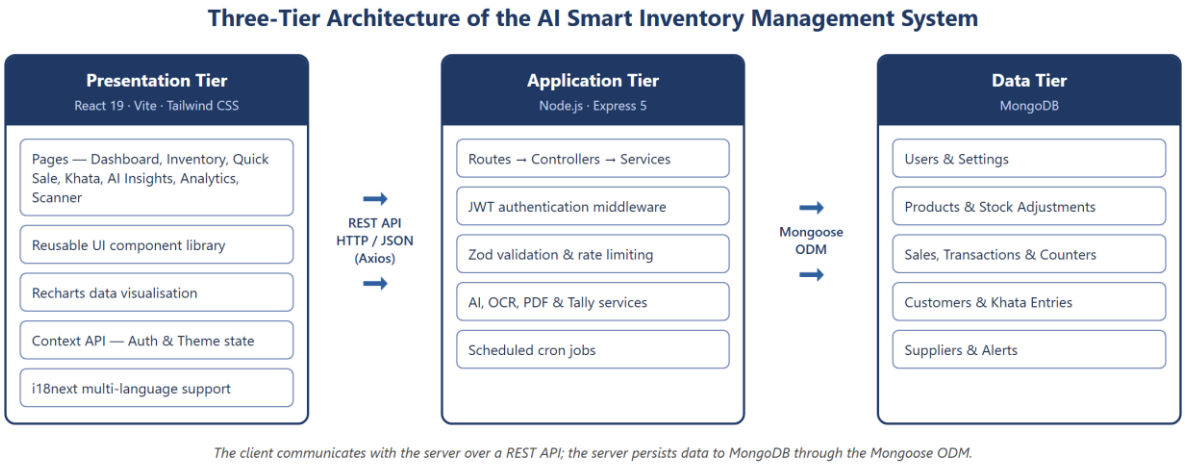

Figure 3 — Three-Tier Architecture of the System

On the server, every request passes through a defined pipeline: authentication middleware verifies the JSON Web Token, validation middleware checks the request body against a Zod schema, the controller orchestrates the operation, and the service layer performs the business logic and database access. A central error-handling middleware converts all failures into consistent JSON error responses.

CHAPTER 2

2. Introduction to Full-Stack Development with JavaScript

Full-stack development refers to the practice of building both the client side (front end) and the server side (back end) of a web application, together with the database that stores its data. A developer or team that handles all of these layers is said to work across the full stack.

Traditionally, the front end and back end of an application were written in different programming languages — for example JavaScript in the browser and PHP, Java or Python on the server. The arrival of Node.js made it possible to run JavaScript on the server as well, which means a single language can be used across the entire application. This is the foundation of the MERN stack used in this project.

MERN is an acronym for four JavaScript-based technologies that together cover every layer of a modern web application:

- **MongoDB** — a NoSQL document database that stores data as flexible JSON-like documents.
- **Express.js** — a minimal and flexible web-application framework for Node.js that is used to build the REST API.
- **React** — a component-based JavaScript library for building interactive user interfaces.
- **Node.js** — a JavaScript runtime that executes the server-side code.

These four technologies and their respective roles are summarised in Figure 4.

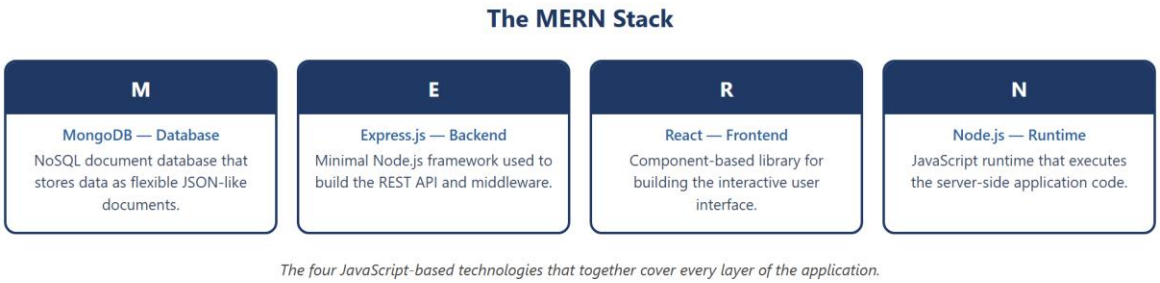

Figure 4 — The MERN Stack

Using JavaScript end-to-end offers several advantages: a single language reduces the learning curve and context-switching; data is exchanged between client and server as JSON, which maps naturally to JavaScript objects and to MongoDB documents; and a large ecosystem of open-source packages is available through the Node Package Manager (npm). These factors make the MERN stack a popular and productive choice for building data-driven applications such as the one developed in this project.

The way these layers are arranged, from the browser down to the database, is shown in Figure 5.

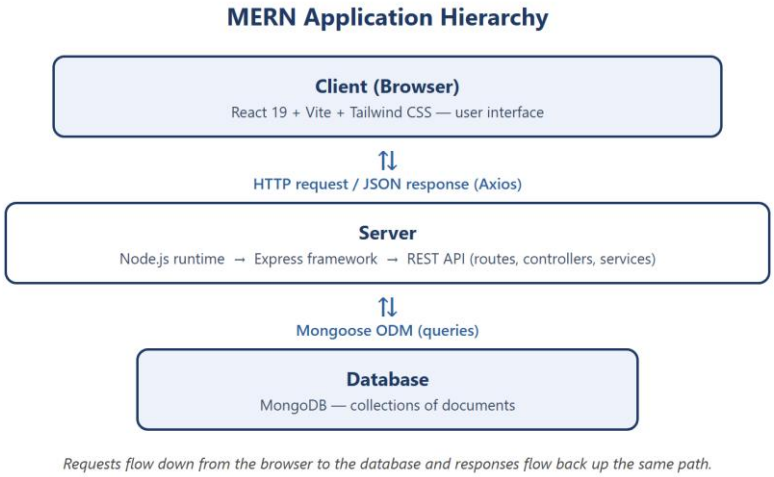

Figure 5 — MERN Application Hierarchy

CHAPTER 3

3. Technologies and Concepts

This chapter describes the technologies, libraries and concepts used to build the AI Smart Inventory Management System, and explains the role each one plays in the application.

The principal technologies used in the project, grouped by the layer in which they operate, are shown in Figure 6.

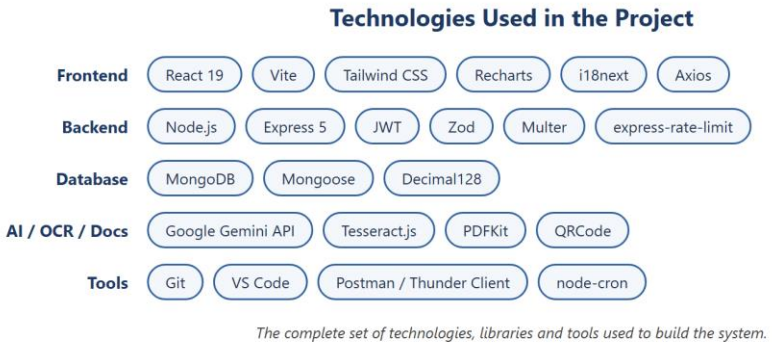

Figure 6 — Technologies Used in the Project

3.1 React JS

React is an open-source JavaScript library, developed by Meta (Facebook), for building user interfaces. It follows a component-based model in which the interface is broken down into small, reusable pieces called components, each managing its own structure and behaviour. React uses a virtual DOM to update only the parts of the page that actually change, which makes the interface fast and responsive. In this project, React (version 19) together with the Vite build tool and the Tailwind CSS framework is used to build the entire front end — the dashboard, inventory, billing, khata, analytics and other pages.

Figure 7 shows a simple React functional component that displays the details of a single product.

A Simple React Component

ProductCard.jsx

```jsx
import React from 'react';

// A reusable card that displays one product
function ProductCard({ product }) {
  return (
    <div className="card">
      <h3>{product.name}</h3>
      <p>Price: ₹{product.price}</p>
      <p>In stock: {product.stockQty}</p>
    </div>
  );
}

export default ProductCard;
```

Figure 7 — A Simple React Component

3.2 React Hooks

Hooks are functions introduced in React that let functional components use state and other React features without writing class components. The most commonly used hooks are useState, which adds local state to a component, and useEffect, which performs side effects such as fetching data when a component renders. This project also uses useContext to consume shared application state and a number of custom hooks (for example, hooks for inventory, suppliers and transactions) that encapsulate data fetching and reuse it across pages.

Figure 8 shows a custom hook that uses the useState and useEffect hooks to load data from the API.

A React Hook Example (useState & useEffect)

useProducts.js

```jsx
import { useState, useEffect } from 'react';

// Custom hook that loads products from the API
function useProducts() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    api.get('/products')
       .then((res) => setProducts(res.data));
  }, []);

  return products;
}

```

Figure 8 — A React Hook Example

3.3 State Management — React Context API

As an application grows, data such as the logged-in user, the selected theme and workspace settings need to be shared across many components. While libraries such as Redux are often used for this purpose in large applications, this project uses React's built-in Context API, which provides a lightweight and sufficient way to manage and share global state without the additional boilerplate of an external library. The application defines several contexts — for authentication, theme, toast notifications and onboarding — each wrapping the component tree and making its data available wherever it is needed.

Figure 9 shows a context provider that stores the authenticated user and exposes login and logout functions to the entire component tree.

A React Context Provider (State Management)

AuthContext.jsx

```jsx
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login  = (data) => setUser(data);
  const logout = () => setUser(null);

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

Figure 9 — A React Context Provider

3.3.1 Reducer Pattern for State Logic

The reducer pattern is a common technique for keeping state transitions predictable: a reducer is a pure function that receives the current state and an action and returns the next state, centralising all state-updating logic in one place. For the relatively simple global state required by this project — the authenticated user, the selected theme and notifications — the Context API is used together with the useState hook, which is sufficient; the reducer pattern (via the useReducer hook) can be introduced for any context whose logic later grows more complex.

3.3.2 Context Provider as the Application Store

A context provider component holds the shared state and exposes it, together with the functions that modify it, to all of its descendant components. The provider therefore plays the same role as a store: it is the single place where a particular slice of global state lives, and any component can read from or update it through the corresponding hook.

The overall flow of state through the Context API is summarised in Figure 10.

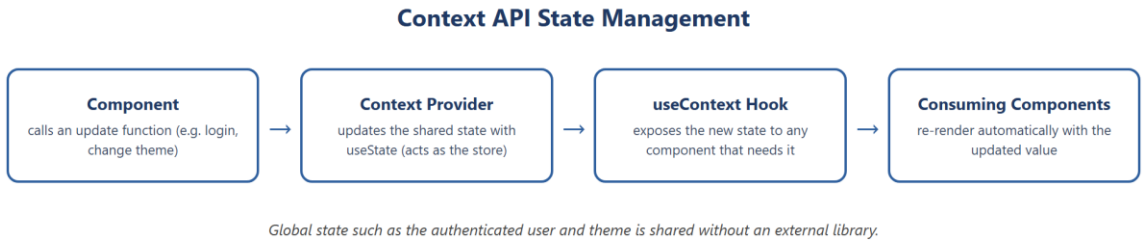

Figure 10 — Context API State Management

3.4 Axios

Axios is a promise-based HTTP client for JavaScript that runs in the browser and in Node.js. It is used on the client to send requests to the server's REST API and to receive responses as JSON. In this project a single configured Axios instance attaches the authentication token to every request and centralises error handling, so that all of the application's service modules communicate with the back end in a consistent way.

3.5 Application Programming Interface (API)

An Application Programming Interface (API) is a defined set of rules that allows two software components to communicate. This project exposes a RESTful API: the server publishes a collection of endpoints, each identified by a URL and an HTTP method (GET, POST, PUT, DELETE), and the client calls these endpoints to read and modify data. All endpoints are versioned under the path /api/v1 and exchange data in JSON format. Representational State Transfer (REST) is an architectural style that treats server data as resources and uses standard HTTP verbs to operate on them, which makes the API predictable and easy to consume.

3.6 Cloud Database — MongoDB Atlas

The application stores its data in MongoDB, which can be hosted locally during development or in the cloud using MongoDB Atlas. Atlas is the official cloud database service for MongoDB; it provides a managed, always-available database with automatic backups and security controls, accessed through a single connection string. Hosting the database in the cloud allows the application to be deployed and accessed from anywhere, and removes the burden of maintaining database server infrastructure.

3.7 Node JS

Node.js is an open-source, cross-platform JavaScript runtime built on Google Chrome's V8 engine. It allows JavaScript to be executed outside the browser and is used to run the server side of the application. Node.js uses an event-driven, non-blocking input/output model, which makes it efficient and well suited to data-intensive applications that handle many simultaneous requests.

Figure 11 shows a minimal web server written using the built-in http module of Node.js.

A Simple Node.js Web Server

server.js

```
1. const http = require('http');
2.
3. // A minimal web server using the built-in http module
4. const server = http.createServer((req, res) => {
5.   res.writeHead(200, { 'Content-Type': 'text/plain' });
6.   res.end('Hello from the Node.js server');
7. });
8.
9. server.listen(5000, () => {
10.   console.log('Server running on port 5000');
11. });
```

Figure 11 — A Simple Node.js Web Server

3.7.1 Features of Node JS

- **Asynchronous and event-driven** — operations do not block the program while waiting for input/output, allowing many requests to be handled concurrently.
- **Single-threaded but scalable** — a single thread with an event loop serves a large number of connections efficiently.
- **Rich package ecosystem** — the Node Package Manager (npm) provides access to a vast library of reusable open-source modules.

- **Cross-platform** — the same code runs on Windows, macOS and Linux.

3.8 Express JS

Express.js is a fast, minimal and flexible web-application framework for Node.js. It simplifies the creation of a web server and a REST API by providing a clean way to define routes, apply middleware and handle requests and responses. In this project Express (version 5) is used to build the entire back-end API, with middleware for authentication, request validation, logging, rate limiting and centralised error handling.

The same web server is considerably simpler to build with Express, as shown in Figure 12.

An Express Server

app.js

```
1. const express = require('express');
2. const app = express();
3.
4. app.use(express.json());
5.
6. // A simple REST route
7. app.get('/api/v1/products', (req, res) => {
8.   res.json({ message: 'List of products' });
9. });
10.
11. app.listen(5000, () => {
12.   console.log('Express server running on port 5000');
13. });
```

Figure 12 — An Express Server

3.9 MongoDB and Mongoose

3.9.1 MongoDB

MongoDB is a NoSQL, document-oriented database. Instead of storing data in tables of rows and columns as a relational database does, it stores data as flexible, JSON-like documents grouped into collections. This flexibility makes it easy to evolve the data model as the application grows, and its document structure maps naturally onto the objects used in JavaScript.

In deployment the database is hosted on a managed MongoDB Atlas cluster, illustrated in Figure 13.

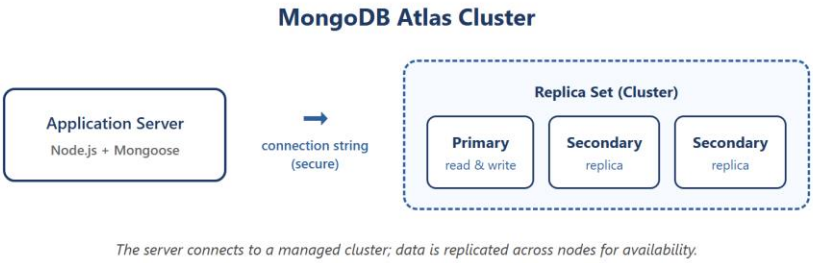

Figure 13 — MongoDB Atlas Cluster

3.9.2 Key Components of MongoDB Architecture

- **Document** — the basic unit of data, stored in a binary JSON format called BSON.
- **Collection** — a group of related documents, analogous to a table in a relational database.
- **Database** — a container that holds one or more collections.
- **Index** — a structure that improves the speed of queries on selected fields.
- **_id field** — a unique identifier automatically assigned to every document, serving as its primary key.

3.9.3 Mongoose

Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js. It allows the developer to define schemas that describe the structure, data types and validation rules of documents, and then work with the data through model objects. In this project Mongoose defines all of the data models — User, Product, Sale, Customer, KhataEntry, Supplier, StockAdjustment and others — and enforces that monetary values are stored using the high-precision Decimal128 type.

3.10 Thunder Client / Postman

Postman and Thunder Client are tools used to test REST APIs. They allow a developer to send HTTP requests to the server's endpoints and inspect the responses without building a front end first. Thunder Client is a lightweight extension that runs inside Visual Studio Code. During development, these tools were used to test each endpoint of the API —

verifying status codes, response structure and the correctness of the GST calculations — before connecting it to the React client.

3.11 Project Implementation

The application is organised into two main parts: a client folder containing the React front end and a server folder containing the Node.js and Express back end. The server is further structured into routes, controllers, services, models, middlewares, validators and scheduled jobs, which keeps the code modular and maintainable. The client is structured into pages, reusable components, services that call the API, custom hooks and context providers. The two parts communicate exclusively through the REST API.

The dependencies of the two parts of the application are declared in their package.json files. Figure 14 lists the principal client-side dependencies and Figure 15 the server-side dependencies, the latter including the AI, OCR, PDF and scheduling libraries that power the system.

Client-Side Dependencies (package.json)

client / package.json

```
{
  "dependencies": {
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "react-router-dom": "^7.13.1",
    "axios": "^1.13.6",
    "recharts": "^3.8.0",
    "react-i18next": "^17.0.6",
    "lucide-react": "^0.577.0",
    "qrcode.react": "^4.2.0"
  },
  "devDependencies": {
    "vite": "^8.0.0",
    "tailwindcss": "^3.4.19"
  }
}
```

Figure 14 — Client-Side Dependencies (package.json)

Server-Side Dependencies (package.json)

server / package.json

```
{
  "dependencies": {
    "express": "^5.2.1",            // web framework
    "mongoose": "^9.3.1",           // MongoDB ODM
    "jsonwebtoken": "^9.0.3",       // JWT auth
    "bcryptjs": "^3.0.3",           // password hashing
    "zod": "^4.3.6",                // validation
    "@google/generative-ai": "^0.24.1", // Gemini AI
    "tesseract.js": "^7.0.0",       // OCR
    "pdfkit": "^0.18.0",            // PDF invoices
    "qrcode": "^1.5.4",             // UPI QR codes
    "node-cron": "^4.2.1",          // scheduled jobs
    "multer": "^2.1.1",             // file uploads
    "decimal.js": "^10.6.0"          // money precision
  }
}
```

Figure 15 — Server-Side Dependencies (package.json)

3.12 Development Environment Setup

The development environment was set up by installing Node.js (which includes the npm package manager), a code editor and access to a MongoDB database. The server and client dependencies are declared in their respective package.json files and installed with npm. During development the React client is served by the Vite development server with hot-module replacement, while the Express server runs as a separate Node.js process; the two communicate over HTTP.

3.13 Version Control System (VCS)

A Version Control System records changes to the source code over time so that specific versions can be recalled later, and so that work can be managed in a disciplined way. This project uses Git, the most widely used distributed version control system. Git tracks the history of every file, allows changes to be grouped into commits with descriptive messages, and supports branching so that new features can be developed in isolation before being merged.

3.14 Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight and extensible source-code editor developed by Microsoft. It offers features such as syntax highlighting, intelligent code completion, integrated debugging, Git integration and a large marketplace of extensions. VS

Code was used as the primary development environment for writing, testing and debugging both the client and server code of this project.

3.15 JSON Web Tokens (JWT)

A JSON Web Token (JWT) is a compact, self-contained and digitally signed token used to securely transmit information between parties. In this application JWTs are used for authentication. When a user logs in with valid credentials, the server verifies the password against its bcrypt hash and issues a signed token. The client stores this token and sends it with every subsequent request; the server's authentication middleware verifies the signature before allowing access to any protected route. Because the token is self-contained, the server does not need to store session state, which keeps the API stateless and scalable. The authentication flow is shown in Figure 16.

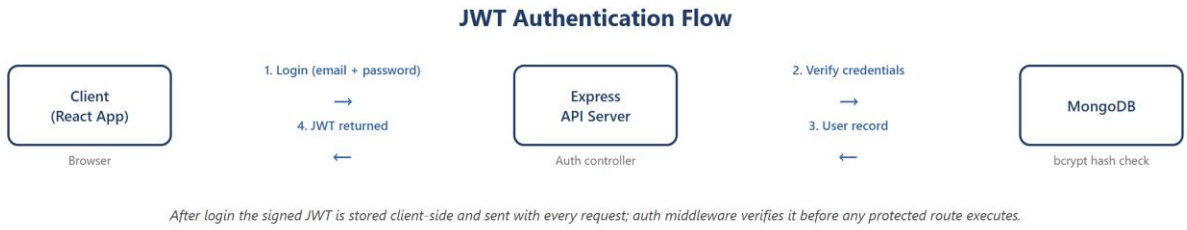

Figure 16 — JWT Authentication Flow

CHAPTER 4

4. Product Overview

This chapter describes the system from the point of view of its users and requirements, and gives an overview of the client and server parts of the application.

4.1 Users and Stakeholders

- **Business owner (primary user)** — manages inventory, performs billing, tracks customer credit, and reviews analytics and AI insights to run the business.
- **Shop staff** — use the billing and inventory screens for day-to-day counter operations.
- **Customers** — benefit indirectly through faster billing, digital invoices, UPI payment and an accurate record of their credit.
- **Suppliers** — represented in the system through supplier records and purchase transactions.
- **Accountant** — consumes the Tally-compatible export of transaction data for bookkeeping and tax filing.

4.2 Functional and Non-Functional Requirements

4.2.1 Functional Requirements

- User registration, login and authentication with secure password storage.
- Creation and management of products, including category, price, GST rate, unit of measure and reorder level.
- Point-of-sale billing with automatic GST computation and PDF invoice generation.
- Recording of payments by cash, UPI/online or credit, with credit posted to the customer ledger.
- Management of customers and a digital khata with running balances and repayments.
- Management of suppliers and recording of stock adjustments (shrinkage).
- Demand forecasting and reorder suggestions derived from sales history.
- An AI help assistant (Google Gemini) and automatic reading of supplier bills (Gemini Vision with a Tesseract.js OCR fallback).
- Analytics dashboard, scheduled report generation and smart stock alerts.

4.2.2 Non-Functional Requirements

- **Security** — authenticated and validated APIs, hashed passwords and protection against abuse.
- **Reliability** — accurate monetary calculations using high-precision decimal arithmetic.
- **Usability** — a simple, multi-language interface suitable for non-technical users.
- **Performance** — a responsive single-page interface that updates efficiently.
- **Maintainability** — a modular, layered codebase that is easy to extend and test.
- **Portability** — a web application accessible from any modern browser.

4.3 Frontend Application (Client Side)

The client side is a single-page application built with React. It presents the user interface through a set of pages — Dashboard, Inventory, Quick Sale, Customers and Khata, Suppliers, AI Insights, Analytics, Scanner, Reports and Settings — organised within a common dashboard layout with a sidebar and top navigation. The client communicates with the server entirely through the REST API using Axios, and manages shared state such as the authenticated user and theme through React context providers.

4.4 Backend Server (Server Side)

The server side is a REST API built with Node.js and Express. It receives requests from the client, authenticates and validates them, executes the required business logic and returns JSON responses. The server also hosts the integrations that give the system its intelligence and automation: an AI help-assistant and insights module, a bill-reading service (Gemini Vision with a Tesseract.js OCR fallback), the PDF-invoice service and the scheduled cron jobs. It persists all data to MongoDB through Mongoose. The principal collections of the database and the relationships between them are shown in Figure 17.

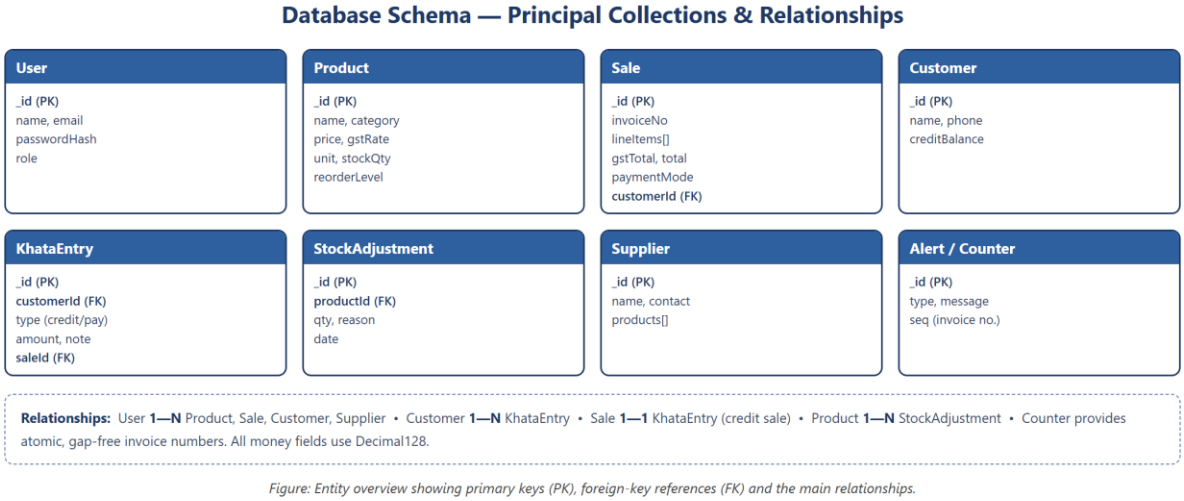

Figure: Entity overview showing primary keys (PK), foreign-key references (FK) and the main relationships.

Figure 17 — Database Schema and Relationships

4.5 Client-Side Modules and Pages

- **Authentication and onboarding** — login, registration and a guided multi-step onboarding wizard for new businesses.
- **Inventory pages** — product listing, creation and editing, low-stock indicators and the shrinkage (stock-adjustment) screen.
- **Quick Sale page** — product search, weight or amount entry, the bill cart and the payment-mode picker.
- **Customers and Khata pages** — customer directory and the digital credit ledger.
- **AI Insights and Analytics pages** — demand forecasts, reorder report and charts rendered with Recharts.
- **Scanner page** — upload of a supplier bill for OCR-based product import.

4.6 Server-Side API and Services

- **Route groups** — modular endpoints for authentication, products, sales, customers, khata, suppliers, inventory adjustments, AI, OCR, analytics, reports and settings under /api/v1.
- **Controllers and services** — controllers handle the HTTP layer while services contain the business logic and database access.

- **Middleware** — JWT authentication, Zod-based validation, rate limiting and central error handling.
- **Integration services** — an AI help-assistant and insights module (rule-based analytics plus a Gemini-powered chatbot), a bill-reading service (Gemini Vision with a Tesseract.js OCR fallback), the PDFKit invoice service and the Tally export service.
- **Scheduled jobs** — a report generator and a smart-alerts job that run automatically using node-cron.

CHAPTER 5

5. Conclusion and Future Work

5.1 Critical Evaluation

The AI Smart Inventory Management System successfully digitises the complete daily workflow of a small retail business — inventory, billing, customer credit, supplier management and analysis — within a single coherent application. Measured against the objectives set out at the start of the project, the system meets each of them: inventory is tracked in real time, billing applies GST automatically and produces a professional invoice, customer credit is maintained digitally, and artificial intelligence is used in a practical rather than decorative way.

A particular strength of the implementation is its engineering discipline: monetary values use high-precision decimal arithmetic to eliminate rounding errors, every API request is validated against a schema, access is protected by token-based authentication, and invoice numbers are generated atomically to remain sequential and gap-free. The layered architecture keeps the codebase modular and testable. The main limitations of the current version are its dependence on an internet connection, the absence of a dedicated mobile application, and support for only a single business account per login — each of which is addressed in the future-work plan below.

5.2 Future Work

1. **Mobile application:** a React Native or progressive web application so that billing can be performed on any phone at the counter.
2. **Messaging integration:** automatic delivery of invoices and credit-payment reminders through WhatsApp or SMS.
3. **Barcode scanning:** camera-based barcode support for rapid billing of packaged goods.
4. **Multi-store and multi-user support:** workspaces with role-based permissions for staff members across multiple branches.
5. **Offline-first operation:** local storage with background synchronisation so that billing continues during internet outages.

6. **Advanced intelligence:** price optimisation, festival-season demand models and anomaly detection for identifying possible theft.

5.3 Final Thoughts

The project demonstrates the full breadth of modern full-stack engineering: requirement analysis, layered system architecture, document database design, secure and validated REST APIs, a contemporary React user interface, the integration of artificial-intelligence and OCR services, scheduled automation, and testing. Beyond the academic exercise, the system addresses a genuine problem faced by millions of small retailers, and its modular design provides a solid foundation on which the enhancements described above can be built. The development of this project has been a valuable opportunity to apply theoretical knowledge to a complete, real-world software product.

REFERENCES

[1] React – A JavaScript Library for Building User Interfaces. https://react.dev

[2] Express.js – Fast, Unopinionated, Minimalist Web Framework for Node.js. https://expressjs.com

[3] MongoDB Documentation. https://www.mongodb.com/docs

[4] Mongoose ODM Documentation. https://mongoosejs.com

[5] Node.js Documentation. https://nodejs.org/en/docs

[6] Google Gemini API (Google AI for Developers). https://ai.google.dev

[7] Tesseract.js – Pure JavaScript OCR. https://tesseract.projectnaptha.com

[8] PDFKit – A JavaScript PDF Generation Library. https://pdfkit.org

[9] Tailwind CSS Documentation. https://tailwindcss.com

[10] JSON Web Tokens. https://jwt.io

[11] Vite – Next Generation Frontend Tooling. https://vitejs.dev

[12] Gujarat Technological University (GTU). https://www.gtu.ac.in